

实现MVVM架构

北京理工大学计算机学院
金旭亮



概述

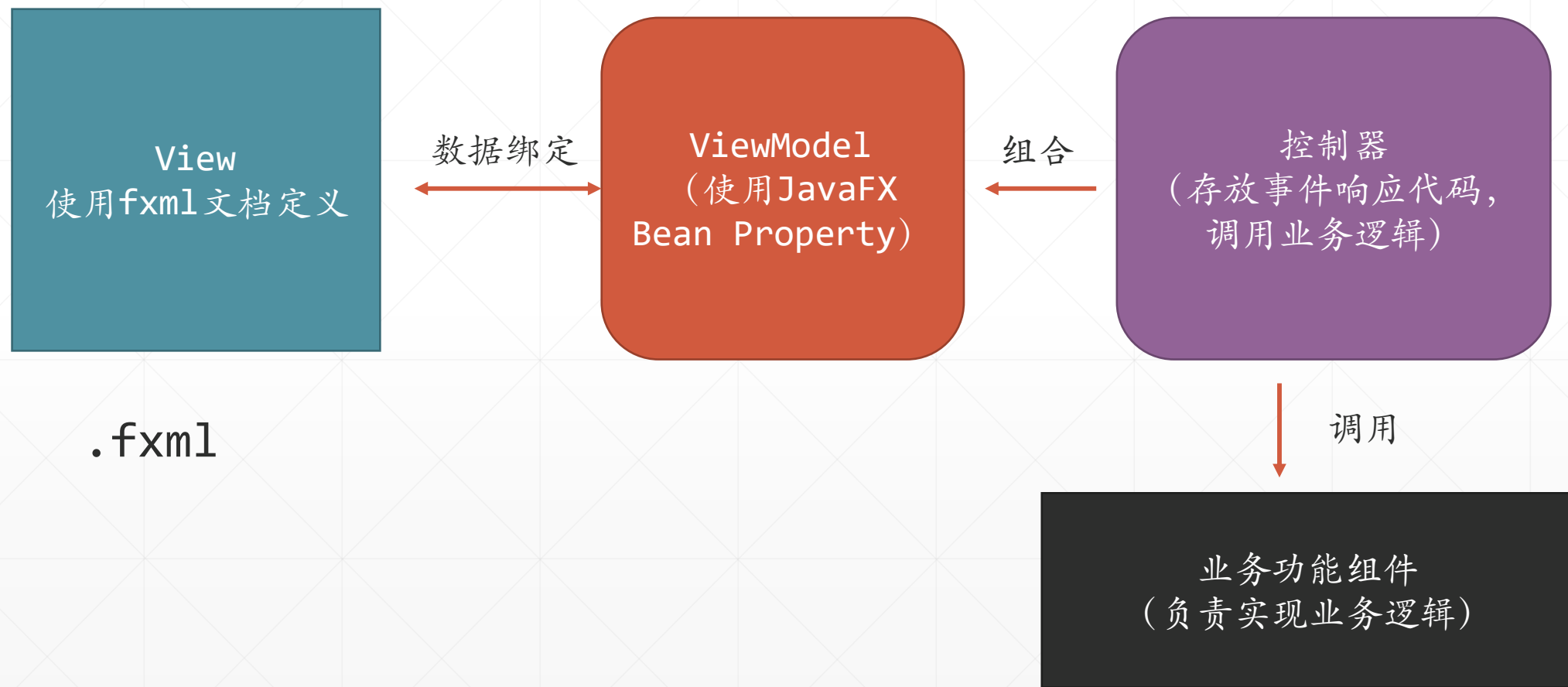


MVVM设计模式，最早在微软的WPF程序中得到广泛应用，后来被迁移到了其它平台和框架中。

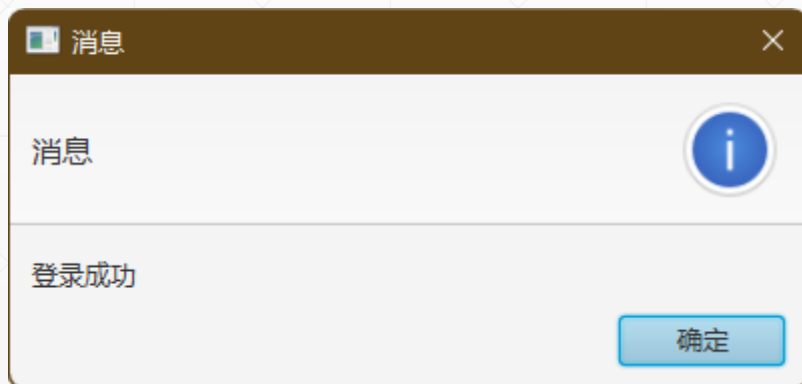


JavaFX与WPF有着相似的技术特性，因此，JavaFX实现MVVM架构也是一件并不复杂的事情。

JavaFX中MVVM架构实现



MVVM示例



示例数据结构

C LoginViewModel		
f	name	StringProperty
f	password	StringProperty
p	name	String
p	password	String

E LoginResult		
m	LoginResult(String)	
f	FAILURE	
f	info	String
f	SUCCESS	
m	toString()	String
m	valueOf(String)	LoginResult
m	values()	LoginResult[]

实现登录处理逻辑



//负责封装用户登录密码校验功能

```
public class LoginManager {
```

```
    1 usage
```

```
    public LoginResult login(String name, String password) {
```

```
        if (name.equals("jxl") && password.equals("123456")) {
```

```
            return LoginResult.SUCCESS;
```

```
        }
```

```
        return LoginResult.FAILURE;
```

```
    }
```

```
}
```

在控制器中“串”起一切

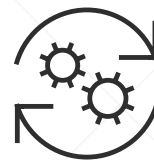
将各种需要的对象和事件响应代码都组合到控制器中，并建立视图与ViewModel对象间的双向绑定关系。

```
@Override
public void initialize(URL url, ResourceBundle resourceBundle) {
    //为按钮挂接事件响应代码
    btnOK.setOnAction(this::login);
    btnCancel.setOnAction(this::cancel);

    //建立View与ViewModel间的数据绑定关系
    txtName.textProperty().bindBidirectional(loginViewModel.nameProperty());
    txtPassword.textProperty().bindBidirectional(loginViewModel.passwordProperty());
}
```

LoginController		
m	LoginController()	
f	btnOK	Button
f	<u>loginViewModel</u>	<u>LoginViewModel</u>
f	txtName	TextField
f	txtPassword	PasswordField
f	btnCancel	Button
f	<u>loginManager</u>	<u>LoginManager</u>
m	initialize(URL, ResourceBundle)	void
m	cancel(ActionEvent)	void
m	login(ActionEvent)	void

调用业务组件所提供的业务处理功能



```
1 usage
private void cancel(ActionEvent actionEvent) {
    //取消时, 重置所有输入控件
    loginViewModel.setName("");
    loginViewModel.setPassword("");
    //焦点回到用户名文本框
    txtName.requestFocus();
}
```

```
1 usage
private void login(ActionEvent actionEvent) {
    if (loginViewModel.getName().trim().length() > 0 &&
        loginViewModel.getPassword().trim().length() > 0) {
        //调用业务组件, 实现“登录”这一业务功能
        var result : LoginResult = loginManager.login(loginViewModel.getName(),
            loginViewModel.getPassword());
        //依据处理结果进行不同的响应
        if (result == LoginResult.SUCCESS) {
            new Alert(Alert.AlertType.INFORMATION, result.toString()).show();
        } else {
            new Alert(Alert.AlertType.ERROR, result.toString()).show();
        }
    } else {
        new Alert(Alert.AlertType.WARNING, contentText: "用户名及密码不能为空",
            ButtonType.OK).show();
    }
}
```


JavaFX应用实现MVVM设计模式小结



在ViewModel中放置JavaFX Bean Properties，它将绑定到UI控件的特定属性，利用JavaFX数据绑定机制实现UI的自动更新。



ViewModel实例被组合到控制器内部，等待与视图中各控件的绑定。



在View以声明方式，或在Controller中以代码方式，建立ViewModel和控件之间的数据绑定关系。



如果使用fxml来定义视图，只需在fxml中使用fx:controller关联上控制器，之后，在控制器中使用@FXML定义字段引用视图上定义的控件，控件就可以使用代码与ViewModel建立起数据绑定关系。